COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 Data Structure and Algorithm | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 10/02/26 | **DATE SUBMITTED:** | 10/06/26 |
| **NAME:** | Nieles, Cendy M, | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 10p1 Graphs ]

## 1. PROCEDURE OUTPUT

What procedures did you perform?

Code

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

ILO A > main.cpp > main()

```cpp
#include <iostream>

// Stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// Structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // Insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // Point new node to current head
        return newNode;
    }

    int N; // Number of nodes in the graph

public:
    adjNode **head; // Adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        head = new adjNode*[N]();
        this->N = N;

        // Initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
```

```
37
38          // Construct directed graph by adding edges to it
39          for (unsigned i = 0; i < n; i++) {
40              int start_ver = edges[i].start_ver;
41              int end_ver = edges[i].end_ver;
42              int weight = edges[i].weight;
43
44              // Insert in the beginning
45              adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
46
47              // Point head pointer to new node
48              head[start_ver] = newNode;
49          }
50      }
51
52      // Destructor
53      ~DiaGraph() {
54          for (int i = 0; i < N; i++) {
55              adjNode* current = head[i];
56              while (current != nullptr) {
57                  adjNode* next = current->next;
58                  delete current;
59                  current = next;
60              }
61          }
62          delete[] head;
63      }
64  };
65
66  // Print all adjacent vertices of given vertex
67  void display_AdjList(adjNode* ptr, int i) {
68      while (ptr != nullptr) {
69          std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
```

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

ILO A > main.cpp > DiaGraph > DiaGraph(graphEdge [], int, int)

```cpp
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

// Graph implementation
int main() {
    // Graph edges array: (x, y, w) -> edge from x to y with weight w
    graphEdge edges[] = {
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

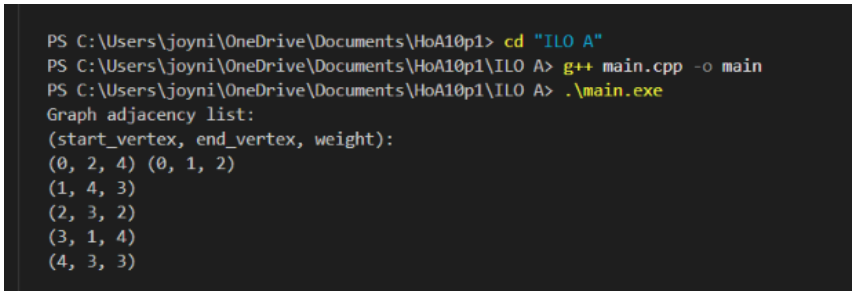
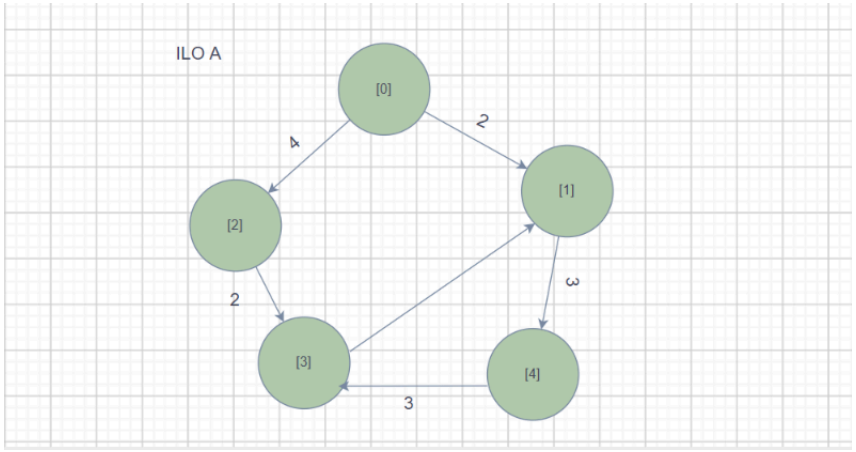
    // Calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // Construct graph
    DiaGraph diagraph(edges, n, N);

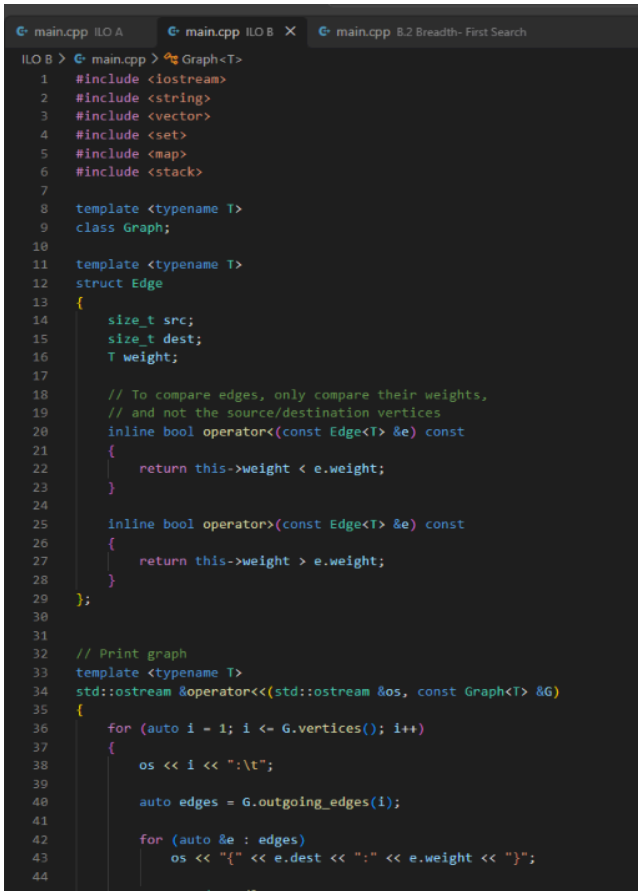
    // Print adjacency list representation of the graph
    std::cout << "Graph adjacency list:" << std::endl
              << "(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++) {
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

| | |
|---|---|
| Output |  |
| Graph |  |

## ILO B
## B.1. Depth-First Search

| | |
|---|---|
| Code |  |

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};


// Print graph
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);

        for (auto &e : edges)
            os << "{" << e.dest << ":" << e.weight << "}";
```


main.cpp ILO A
main.cpp ILO B
main.cpp B.2 Breadth- First Search
ILO B > main.cpp > Graph<T>
34 std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
36 for (auto i = 1; i <= G.vertices(); i++)
45 os << std::endl;
46 }
47
48 return os;
49 }
50
51
52 template <typename T>
53 class Graph
54 {
55 public:
56
57 // Initialize the graph with N vertices
58 Graph(size_t N) : V(N)
59 {
60 }
61
62 // Return number of vertices in the graph
63 auto vertices() const
64 {
65 return V;
66 }
67
68 // Return all edges in the graph
69 auto &edges() const
70 {
71 return edge_list;
72 }
73
74 void add_edge(Edge<T> &&e)
75 {
76 // Check if the source and destination vertices are within range
77 if (e.src >= 1 && e.src <= V &&
78 e.dest >= 1 && e.dest <= V)
79 {
80 edge_list.emplace_back(e);
81 }
82 else
83 std::cerr << "Vertex out of bounds" << std::endl;
84 }
85
86 // Return all outgoing edges from vertex v

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

ILO B > main.cpp > Graph<T>

```
53  class Graph
55  public:
87      auto outgoing_edges(size_t v) const
88      {
89          std::vector<Edge<T>> edge_from_v;
90
91          for (auto &e : edge_list)
92          {
93              if (e.src == v)
94                  edge_from_v.emplace_back(e);
95          }
96
97          return edge_from_v;
98      }
99
100     // Overloads the << operator so a graph can be
101     // written directly to a stream
102     // Can be used as std::cout << obj << std::endl;
103     template <typename U>
104     friend std::ostream &operator<<(
105         std::ostream &os,
106         const Graph<U> &G);
107
108 private:
109     size_t V; // Store number of vertices in graph
110     std::vector<Edge<T>> edge_list;
111 };
112
113
114 template <typename T>
115 auto depth_first_search(const Graph<T> &G, size_t dest)
116 {
117     std::stack<size_t> stack;
118     std::vector<size_t> visit_order;
119     std::set<size_t> visited;
120
121     stack.push(1); // Assume that DFS always starts from vertex ID 1
122
123     while (!stack.empty())
124     {
125         auto current_vertex = stack.top();
126         stack.pop();
127
128         // If the current vertex hasn't been visited in the past
```

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

ILO B > main.cpp > Graph<T>

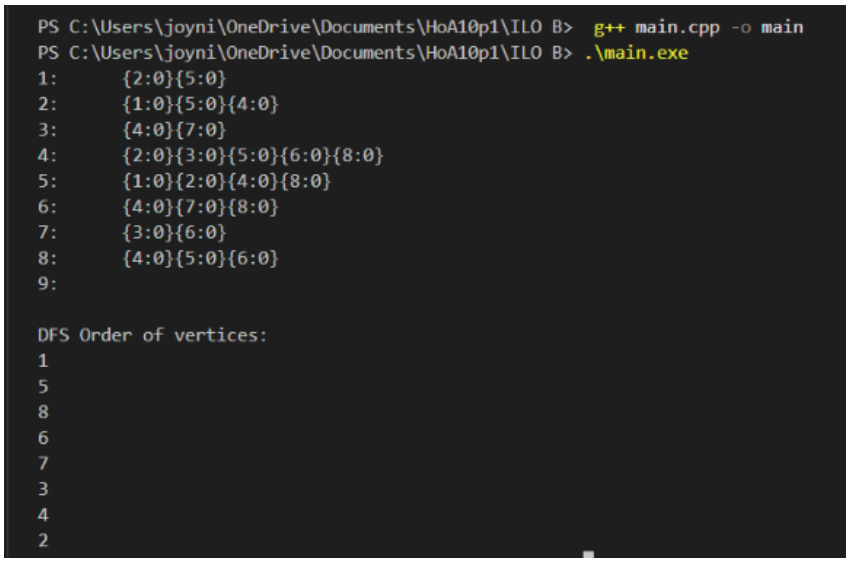
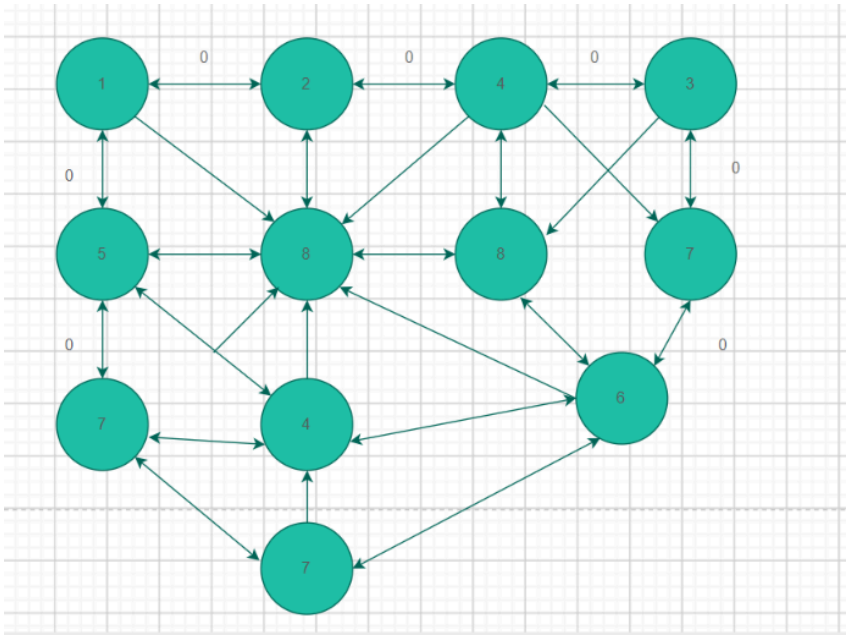
```
115  auto depth_first_search(const Graph<T> &G, size_t dest)
123      while (!stack.empty())
128          // If the current vertex hasn't been visited in the past
129          if (visited.find(current_vertex) == visited.end())
130          {
131              visited.insert(current_vertex);
132              visit_order.push_back(current_vertex);
133
134              for (auto e : G.outgoing_edges(current_vertex))
135              {
136                  // If the vertex hasn't been visited,
137                  // insert it in the stack.
138                  if (visited.find(e.dest) == visited.end())
139                  {
140                      stack.push(e.dest);
141                  }
142              }
143          }
144      }
145
146      return visit_order;
147  }
148
149
150  template <typename T>
151  auto create_reference_graph()
152  {
153      Graph<T> G(9);
154
155      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
156
157      edges[1] = {{2, 0}, {5, 0}};
158      edges[2] = {{1, 0}, {5, 0}, {4, 0}};
159      edges[3] = {{4, 0}, {7, 0}};
160      edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
161      edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
162      edges[6] = {{4, 0}, {7, 0}, {8, 0}};
163      edges[7] = {{3, 0}, {6, 0}};
164      edges[8] = {{4, 0}, {5, 0}, {6, 0}};
165
166      for (auto &i : edges)
167          for (auto &j : i.second)
168              G.add_edge(Edge<T>{i.first, j.first, j.second});
169
```

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

ILO B > main.cpp > Graph<T>

```
53  class Graph
55  public:
87      auto outgoing_edges(size_t v) const
88      {
89          std::vector<Edge<T>> edge_from_v;
90
91          for (auto &e : edge_list)
92          {
93              if (e.src == v)
94                  edge_from_v.emplace_back(e);
95          }
96
97          return edge_from_v;
98      }
99
100     // Overloads the << operator so a graph can be
101     // written directly to a stream
102     // Can be used as std::cout << obj << std::endl;
103     template <typename U>
104     friend std::ostream &operator<<(
105         std::ostream &os,
106         const Graph<U> &G);
107
108 private:
109     size_t V; // Store number of vertices in graph
110     std::vector<Edge<T>> edge_list;
111 };
112
113
114 template <typename T>
115 auto depth_first_search(const Graph<T> &G, size_t dest)
116 {
117     std::stack<size_t> stack;
118     std::vector<size_t> visit_order;
119     std::set<size_t> visited;
120
121     stack.push(1); // Assume that DFS always starts from vertex ID 1
122
123     while (!stack.empty())
124     {
125         auto current_vertex = stack.top();
126         stack.pop();
127
128         // If the current vertex hasn't been visited in the past
```

```
158     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
159     edges[3] = {{4, 0}, {7, 0}};
160     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
161     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
162     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
163     edges[7] = {{3, 0}, {6, 0}};
164     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
165
166     for (auto &i : edges)
167         for (auto &j : i.second)
168             G.add_edge(Edge<T>{i.first, j.first, j.second});
169
170     return G;
171 }
172
173
174 template <typename T>
175 void test_DFS()
176 {
177     // Create an instance of and print the graph
178     auto G = create_reference_graph<unsigned>();
179
180     std::cout << G << std::endl;
181
182     // Run DFS starting from vertex ID 1 and print the order
183     // in which vertices are visited.
184     std::cout << "DFS Order of vertices: " << std::endl;
185
186     auto dfs_visit_order = depth_first_search(G, 1);
187
188     for (auto v : dfs_visit_order)
189         std::cout << v << std::endl;
190 }
191
192
193 int main()
194 {
195     using T = unsigned;
196
197     test_DFS<T>();
198
199     return 0;
200 }
```

| | |
|---|---|
| Output |  |
| Graph |  |

ILO B
B.2. Breadth-First Search

Code

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

B.2 Breadth- First Search > main.cpp > main()

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);

        for (auto &e : edges)
        {
```

```
39          {
40              os << "{" << e.dest << ":" << e.weight << "} ";
41          }
42
43          os << std::endl;
44      }
45
46      return os;
47  }
48
49  template <typename T>
50  class Graph
51  {
52  public:
53      Graph(size_t N) : V(N)
54      {
55      }
56
57      auto vertices() const
58      {
59          return V;
60      }
61
62      const auto &edges() const
63      {
64          return edge_list;
65      }
66
67      void add_edge(Edge<T> &&e)
68      {
69          if (e.src >= 1 && e.src <= V &&
70              e.dest >= 1 && e.dest <= V)
71          {
72              edge_list.emplace_back(e);
73          }
74          else
```

```
        {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;

        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }

        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os,
                                    const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
```

B.2 Breadth- First Search > main.cpp > main()

```
105  auto create_reference_graph()
109      std::map<unsigned, std::vector<std::pair<size_t, T>>> edge
110
111      edges[1] = {{2, 2}, {5, 3}};
112      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
113      edges[3] = {{4, 2}, {7, 3}};
114      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
115      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
116      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
117      edges[7] = {{3, 3}, {6, 4}};
118      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
119
120      for (auto &i : edges)
121      {
122          for (auto &j : i.second)
123          {
124              G.add_edge(
125                  Edge<T>{
126                      i.first,
127                      j.first,
128                      j.second});
129          }
130      }
131
132      return G;
133  }
134
135  template <typename T>
136  auto breadth_first_search(const Graph<T> &G, size_t dest)
137  {
138      std::queue<size_t> queue;
139
140      std::vector<size_t> visit_order;
141      std::set<size_t> visited;
142
143      // Assume that BFS always starts from vertex ID 1
144      queue.push(1);
145
```

main.cpp ILO A | main.cpp ILO B | main.cpp B.2 Breadth- First Search

B.2 Breadth- First Search > main.cpp > main()

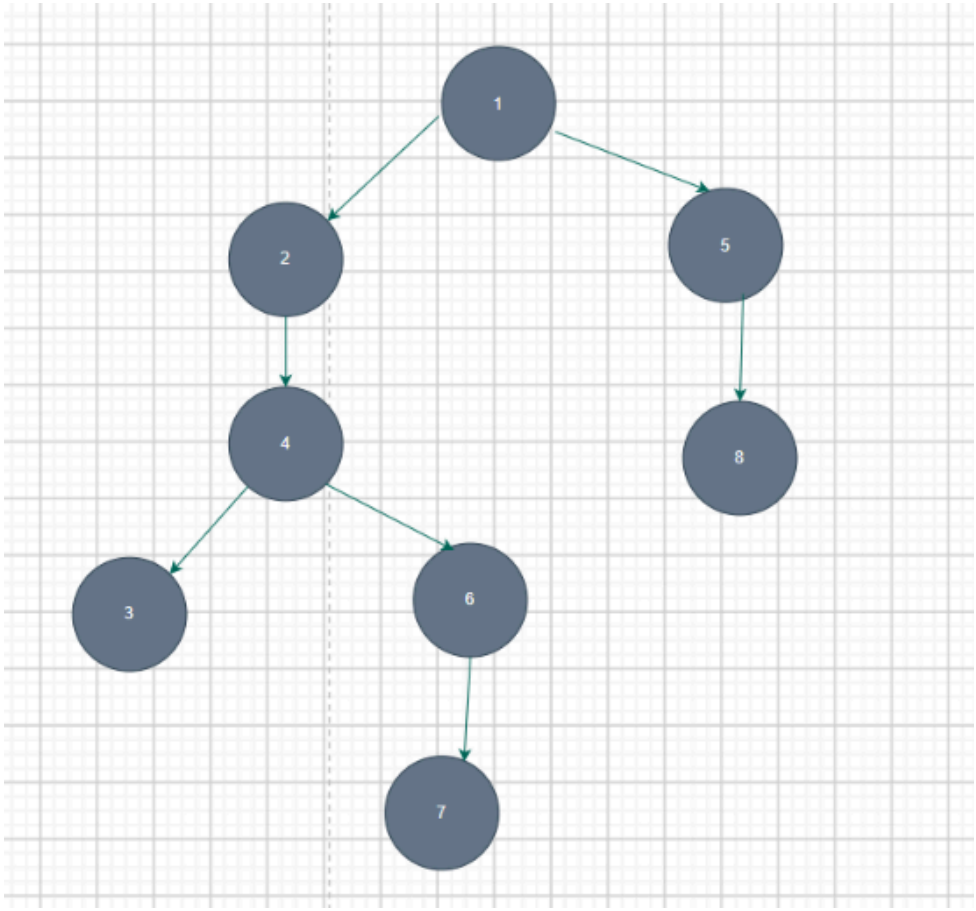
```
136  auto breadth_first_search(const Graph<T> &G, size_t dest)
145
146      while (!queue.empty())
147      {
148          auto current_vertex = queue.front();
149          queue.pop();
150
151          // If the current vertex hasn't been visited in the past
152          if (visited.find(current_vertex) == visited.end())
153          {
154              visited.insert(current_vertex);
155
156              visit_order.push_back(current_vertex);
157
158              for (auto e : G.outgoing_edges(current_vertex))
159              {
160                  queue.push(e.dest);
161              }
162          }
163      }
164
165      return visit_order;
166  }
167
168  template <typename T>
169  void test_BFS()
170  {
171      // Create an instance of and print the graph
172      auto G = create_reference_graph<T>();
173
174      std::cout << "Graph:" << std::endl;
175      std::cout << G << std::endl;
176
177      // Run BFS starting from vertex ID 1 and print the order
178      // in which vertices are visited.
179      std::cout << "BFS Order of vertices:" << std::endl;
180
181      auto bfs_visit_order = breadth_first_search(G, 1);
182
```

```
183     for (auto v : bfs_visit_order)
184     {
185         std::cout << v << " ";
186     }
187
188     std::cout << std::endl;
189 }
190
191 int main()
192 {
193     using T = unsigned;
194
195     test_BFS<T>();
196
197     return 0;
198 }
```

Output

```
PS C:\Users\joyni\OneDrive\Documents\HoA10p1\ILO B> g++ main.cpp -o main
PS C:\Users\joyni\OneDrive\Documents\HoA10p1\ILO B> .\main.exe
1:      {2:0}{5:0}
2:      {1:0}{5:0}{4:0}
3:      {4:0}{7:0}
4:      {2:0}{3:0}{5:0}{6:0}{8:0}
5:      {1:0}{2:0}{4:0}{8:0}
6:      {4:0}{7:0}{8:0}
7:      {3:0}{6:0}
8:      {4:0}{5:0}{6:0}
9:

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\joyni\OneDrive\Documents\HoA10p1\ILO B>
```




**7. Supplementary Activity**

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The most appropriate algorithm is Depth-First Search (DFS). DFS visits a vertex and continues exploring adjacent vertices until it reaches the end of a path. It then backtracks to the previous vertex and explores the remaining unvisited vertices. This method ensures that all locations on the map are visited systematically, making it suitable for the person's goal of visiting every vertex before finishing the traversal.

2.The equivalent of DFS in tree traversal is Preorder, Inorder, and Postorder traversal. These methods explore one branch of the tree as deeply as possible before backtracking. Preorder visits the root first, Inorder visits the root between the left and right subtrees, and Postorder visits the root last. All of them follow the same depth-first strategy.

. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question,provide a graphical comparison, examine pseudocode and code implementation.

The equivalent of DFS in tree traversal is Preorder, Inorder, and Postorder traversal. These methods explore one branch of the tree as deeply as possible before backtracking. Preorder visits the root first, Inorder visits the root between the left and right subtrees, and Postorder visits the root last. All of them follow the same depth-first strategy.

3.In the code, what data structure is used to implement the Breadth First Search?

The Queue data structure is used to implement Breadth-First Search (BFS).A queue follows the First-In, First-Out (FIFO) principle, meaning the first vertex inserted into the queue is the first one to be processed. In BFS, vertices are visited level by level. When a vertex is visited, all its adjacent vertices are added to the queue. The algorithm then removes vertices from the front of the queue and continues until all reachable vertices have been visited.

4.How many times can a node be visited in the BFS?

In Breadth-First Search (BFS), a node is normally visited only once.

When a node is first visited, it is marked as visited and added to the queue. If the algorithm encounters the same node again through another path, it will not visit it again because it has already been marked as visited.

**Conclusion:**

During the implementation and testing of the Depth-First Search (DFS) program, several errors and challenges were encountered. These errors included compilation issues, incorrect commands in the terminal, syntax mistakes, and difficulties in understanding how the graph traversal worked. Through careful debugging, each error was identified and corrected by reviewing the code, checking the logic of the program, and testing the output multiple times. The debugging process helped improve my understanding of C++ programming concepts such as templates, classes, structures, vectors, stacks, and graph representations. It also showed the importance of paying attention to details, such as proper syntax, correct file names during compilation, and accurate implementation of algorithms. After resolving the errors, the program successfully performed the Depth-First Search traversal and produced the expected output. This experience demonstrated that debugging is an essential part of programming because it helps programmers identify problems, improve code quality, and gain a deeper understanding of how algorithms work. Although some parts of the code were challenging to understand at first, the process of analyzing and fixing errors increased my confidence in working with graph algorithms and strengthened my problem-solving skills. Overall, completing this activity provided

valuable knowledge about DFS traversal, graph implementation, and the importance of patience and persistence when debugging programs.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini and Grammarly**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*
*D*

**Purpose of Use:** **For Gemini i used that to guide me in some parts that this activity i dont really understand**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **For Gemini i used that because some of part of the lesson here not really and for grammarly for assist my spelling and pronunciation and my translator..**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY